

Event Registration System

Software Engineering & Architectural Design Report

Package Target: com.vityarthi.eventapp

Runtime Architecture: Java Swing / SQLite

1. Executive Summary

The **Event Registration System** (eventapp) is an offline desktop application designed to streamline event configuration, attendee registration, ticket issuance, and venue check-in verification. Engineered in Java using an N-tier layered architecture, the software integrates a graphical user interface (GUI) powered by Java Swing with a local SQLite embedded database (events.db).

The system provides a self-contained, zero-configuration solution for academic institutions, workshop organizers, and local event managers who require reliable venue entry management without relying on continuous cloud connectivity or external database server hosting.

2. Problem Statement & Project Objectives

Problem Statement

Traditional event management relies either on manual paper logging—which introduces high error rates and severe check-in delays—or complex cloud-hosted platforms that demand constant internet access and complex infrastructure setups. Small-to-medium events require an immediate, lightweight, and offline-first solution that ensures data integrity and enforces strict capacity rules.

Core Objectives

- Automated Capacity Enforcement:** Guarantee that event registrations never exceed pre-configured venue limits.
- Rapid Venue Check-In:** Prevent duplicate entry attempts through real-time ticket ID lookup and status validation.
- Portable Architecture:** Deliver a fully self-contained desktop runtime using an embedded database engine that requires no manual server configuration.
- Maintainable Codebase:** Adhere to object-oriented software engineering principles by decoupling data access, business logic, and presentation tiers.

3. Technology Stack & Technical Requirements

Component	Technology / Library	Specification / Version	Role in Application
Runtime	Java Development Kit (JDK)	JDK 8 or higher	Core execution engine
Presentation	Java Swing / AWT	Standard Java API	Graphical UI components and event handling
Database Engine	SQLite 3	Embedded DB (events.db)	Offline transactional storage
Database Driver	SQLite JDBC	sqlite-jdbc-3.45.1.0.jar	Connectivity bridge between DAO and SQLite
Logging Framework	SLF4J Framework	slf4j-api-2.0.12.jar	Structured runtime logging and error tracking
Build System	Apache Maven	pom.xml	Automated dependency and lifecycle management

4. System Architecture & Component Design

The application is structured around a four-tier architecture under the `com.vityarthi.eventapp` namespace, enforcing strict boundary separation across modules.

```
com.vityarthi.eventapp
├── dao/           # Data Access Layer (DB Connection & Raw SQL)
│   ├── DBConnection.class
│   ├── EventDAO.class
│   └── TicketDAO.class
├── model/        # Domain Entities (Plain Old Java Objects)
│   ├── Event.class
│   └── Ticket.class
├── service/      # Core Business Logic & Operation Results
│   ├── TicketService.class
│   ├── TicketService$RegistrationResult.class
│   └── TicketService$CheckInResult.class
└── ui/          # Graphical Interface & Event Handlers
    ├── MainApp.class
    └── MainUI.class
```

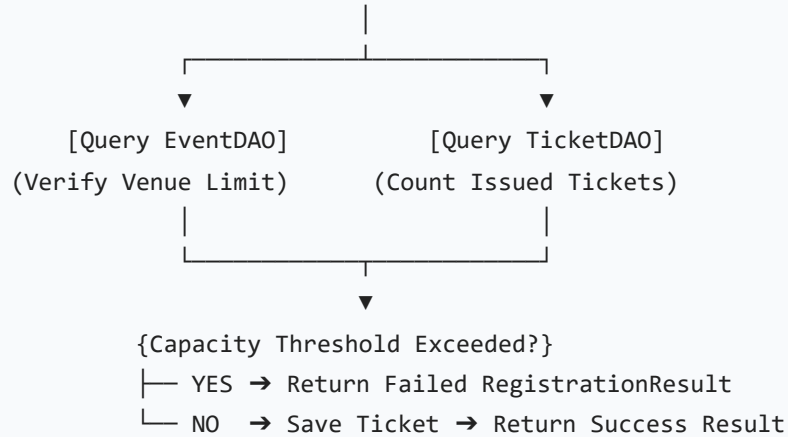
Module Responsibilities

- **Data Access Object (DAO) Layer:** Manages database connections via `DBConnection` and executes parameterized SQL operations inside `EventDAO` and `TicketDAO`.
- **Domain Model Layer:** Standard POJOs (`Event`, `Ticket`) encapsulating entity state, attributes, and data constraints.
- **Service Layer:** Encapsulates core business logic in `TicketService`, validating event capacity before ticket creation and verifying ticket status during venue check-in.
- **Presentation Layer:** Constructed via `MainUI` using Java Swing components, backed by `MainApp` as the central application launcher.

5. Operational Workflows

1. Ticket Registration Workflow

[User Input Form] → [MainUI] → [TicketService.register()]



2. Venue Check-In Verification

- **Step 1:** The venue staff enters the ticket identifier into the check-in module interface.
- **Step 2:** `TicketService.checkIn(ticketId)` queries `TicketDAO` to retrieve ticket record state.
- **Step 3:** If the ticket ID does not exist, an `INVALID_TICKET` result is returned.
- **Step 4:** If the ticket is marked as already used, an `ALREADY_CHECKED_IN` status is triggered to prevent duplicate entry.
- **Step 5:** If valid, the ticket status updates to checked-in with an execution timestamp.

6. System Evaluation & Future Enhancements

Technical Advantages

- **Zero External Infrastructure:** Embedded database engine removes reliance on network servers or cloud connectivity.
- **Structured Response DTOs:** Custom operation result objects prevent unhandled GUI crashes and ensure clear user feedback.

Future Roadmap

- **Asynchronous Thread Management:** Integrate `SwingWorker` threads for off-EDT database queries.
- **PDF Ticket Generation:** Implement PDF export capabilities for printable attendee credentials.
- **Modernized Theme Integration:** Implement `FlatLaf` look-and-feel themes for updated desktop UI aesthetics.